

ONLINE VOTING SYSTEM

Project Report Submitted to

St. Joseph's College (Autonomous)

Affiliated to Bharathidasan University, Tiruchirappalli

in partial fulfilment of the requirement for the award of the

degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

Submitted by

JOHN DIVINE MATHEW J

(Reg.No:23UCS508)

Under the guidance of

Dr. K. SUBASH M.Sc., M.Phil Computer., Ph.D.

Assistant Professor of Computer Science



DEPARTMENT OF COMPUTER SCIENCE

St.JOSEPH'S COLLEGE (AUTONOMOUS)

Accredited at A⁺⁺ Grade (Cycle-IV) by NAAC

Special Heritage Status Awarded by UGC

TIRUCHIRAPPALLI-620 002

APRIL – 2026

Dr. K. SUBASH M.Sc., M.Phil Computer., Ph.D.
Assistant Professor
Department of Computer Science
St. Joseph's College (Autonomous)
Tiruchirappalli-620 002



CERTIFICATE

This is to Certify that the project work entitled **ONLINE VOTING SYSTEM** submitted to the Department of Computer Science, St. Joseph's College (Autonomous), Tiruchirappalli-2, in partial fulfilment for the award of the degree of BACHELOR OF SCIENCE IN COMPUTER SCIENCE, is an original work done by **JOHN DIVINE MATHEW J (Reg.No:23UCS508)**, under my supervision and guidance during the academic period **November 2025 to April 2026**

Signature of the Coordinator

DR. B. Rex Cyril

Assistant Professor and Coordinator,
Department of Computer Science,
St. Joseph's College (Autonomous),
Tiruchirappalli-620 002.

Signature of the Guide

Dr. K. SUBASH

Assistant Professor and Coordinator,
Department of Computer Science,
St. Joseph's College (Autonomous),
Tiruchirappalli-620 002.

Submitted for the viva-voce examination on _____.

Signature of the External Examiner
Date:

Signature of the Internal Examiner
Date:

Mr. JOHN DIVINE MATHEW J
Reg. No. 23UCs508
Department of Computer Science
St. Joseph's College (Autonomous)
Tiruchirappalli-620 002



DECLARATION

I hereby declare that the project work entitled **ONLINE VOTING SYSTEM** is a record of original work done by me during the period of study, under the guidance of **Dr. K. SUBASH M.Sc., M.Phil Computer., Ph.D.** Department of Computer Science, St. Joseph's College, Tiruchirappalli -2. I further declare that any part of this project work has not been submitted elsewhere for the award of any degree or diploma at any other University or Research institute.

Signature of the candidates

ACKNOWLEDGEMENT

I thank and praise God almighty for his immense Grace and Blessing, which guided me in taking up this project work and gave me the confidence and sustained ability to complete this work successfully.

It is my Privilege and pleasure to express my sincere gratitude to **Rev. Dr. S. Mariadoss, SJ Principle** and **Dr. D. Kumar B.A., M.A., M.Phil., B.Ed., DPMIR., Ph.D., M.B.A.,** Deputy Principal of St. Joseph's College (Autonomous) Tiruchirappalli-2 Who gave me a chance for the bachelor's degree.

My sincere gratitude to **Dr. A. Anthony Elderred, M.Sc., M.Phil., Ph.D.,** Dean, School of Computer Science, St. Joseph's College (Autonomous), Tiruchirappalli -2, for motivating me to be a positive personality.

I take immense pleasure to offer my thanks to **Dr. S. Britto Ramesh Kumar, MCA., M.Pil., Ph.D.,** Head, Department of Computer science, St. Joseph's College, Tiruchirappalli-2, for providing me, the precious opportunity to undertake this project work.

In a special manner I thank to **Dr. B. Rex Cyril, M.Sc., B.Ed., M.Phil., SET., Ph.D.,** Coordinator (shift-II), department of Computer Science, St. Joseph's College (Autonomous), Tiruchirappalli-2, for encouraging me in every step.

With a deep sense of Gratitude, I own my profound thanks to my respectful guide **Dr. K. SUBASH M.Sc., M.Phil Computer., Ph.D.** and my class In – Charge **Dr. J. ANTONY JOHN PRABU MCA,MCTS,M.Phil.,Ph.D.** Department of Computer science, St. Joseph's College (Autonomous), Tiruchirappalli-2,for his valuable guidance, inspiration and support for the inception of this Major Project Work.

I am extremely thankful to all staff member of the Computer Science Department, who had provided me the knowledge in carrying out this work.

JOHN DIVINE MATHEW J

23UCS508

TABLE OF CONTENTS

CH. NO.	CONTENTS	PAGE NO.
1	INTRODUCTION 1.1 About the Project	
2	SYSTEM STUDY 2.1 Existing System 2.2 Limitations of Existing System 2.3 Proposed System 2.4 Advantages of Proposed System 2.5 Problem Definition and Description	
3	SYSTEM ANALYSIS 3.1 Packages Selected 3.2 Resources Required	
4	SYSTEM DESIGN 4.1 Architectural Design 4.2 I/O Form Design 4.3 Table Design 4.4 Data Flow Diagram 4.5 Normalization 4.6 Entity Relationship Diagram 4.7 Data Dictionary	
5	SYSTEM DEVELOPMENT 5.1 Functional Documentation 5.2 Special Features of the Language	
6	TESTING 6.1 Types of Testing Done	
7	USER MANUAL 7.1 Installation Procedure	
8	CONCLUSION 8.1 Summary of the Project 8.2 Future Enhancements	
	BIBLIOGRAPHY Website References Book References	
	APPENDIX (a) Program Coding (b) Screenshots	

INTRODUCTION

1.1 About the Project

In modern democratic societies, elections play a vital role in selecting leaders and representatives. Traditional voting methods such as paper ballots and manual vote counting require significant time, manpower, and resources. These systems may also face challenges such as human errors, vote manipulation, and delays in result declaration.

The Online Voting System is a web-based application designed to conduct elections digitally through a secure platform. This system allows registered voters to cast their votes through an online interface while ensuring transparency, accuracy, and security during the election process.

The main objective of the Online Voting System is to provide a secure and user-friendly environment for conducting elections. The system includes multiple modules such as voter registration, nominee management, vote casting, and result generation. An administrator manages the election process by adding candidates, monitoring voter activity, and publishing election results.

The system ensures that each voter can vote only once by using authentication mechanisms. Once the voter logs into the system, they can view the list of nominees and cast their vote, which is securely stored in the database to prevent duplicate voting.

By implementing an online voting platform, the election process becomes faster, more transparent, and easier to manage. The system reduces manual work, minimizes errors, and enables quick result calculation. This project demonstrates how digital technology can improve the efficiency and reliability of the voting process.

SYSTEM STUDY

2.1 Existing System

In the traditional voting system, elections are conducted using paper ballots or electronic voting machines. Voters must physically visit polling stations to cast their votes, and the election process requires a large amount of manpower and resources. After the voting process is completed, votes are counted manually or through machines, which may take a significant amount of time.

In many institutions and small organizations, voting is still conducted manually using ballot papers or simple digital tools. This process may lead to errors, delays in result calculation, and difficulty in managing voter data. Additionally, maintaining transparency and security during the election process becomes challenging when the system is not fully automated.

2.2 Limitations of Existing System

Although traditional voting methods are widely used, they have several limitations:

- Requires physical presence of voters at polling stations
- Time-consuming vote counting process
- Higher chances of human errors
- Requires large manpower and resources
- Difficulty in managing voter and candidate data
- Possibility of vote manipulation or duplicate voting
- Delay in announcing election results

2.3 Proposed System

The proposed system is an **Online Voting System** that allows elections to be conducted digitally through a web-based platform. In this system, registered voters can log in securely and cast their votes online. The system maintains a database to store voter information, candidate details, and voting records.

The administrator manages the election process by adding nominees, verifying voters, monitoring votes, and publishing results. The system ensures that each voter can vote only once and that all votes are recorded securely. The proposed system reduces manual work and provides a faster and more efficient election management process.

2.4 Advantages of Proposed System

The proposed Online Voting System provides several advantages:

- Faster voting and result calculation
- Reduced manual effort and paperwork
- Secure authentication for voters
- Prevention of duplicate voting
- Easy management of voters and candidates
- Increased transparency in election process
- User-friendly interface for voters and administrators

2.5 Problem Definition and Description

In traditional voting systems, conducting elections requires significant time, manpower, and resources. Manual vote counting can lead to delays in result declaration and may also introduce errors in the election process. Additionally, managing voter records and ensuring transparency during elections becomes difficult when the process is not automated.

The main problem addressed in this project is to develop a secure and efficient digital platform for conducting elections. The **Online Voting System** aims to simplify the voting process by allowing voters to cast their votes online while ensuring security, accuracy, and transparency. The system helps administrators manage election activities efficiently and provides faster result generation.

SYSTEM ANALYSIS

System analysis is the process of studying and understanding the requirements of a system before its development. It helps in identifying the system requirements, selecting appropriate technologies, and determining the resources needed for the successful implementation of the project. The main objective of system analysis is to ensure that the system meets user requirements and performs efficiently.

In the **Online Voting System**, system analysis focuses on selecting suitable software technologies, development tools, and hardware resources required to implement the application. The analysis helps in designing a system that is secure, reliable, and easy to use for both voters and administrators.

3.1 Packages Selected

The development of the Online Voting System requires several software packages and technologies that support web application development. These packages provide tools for designing the user interface, handling backend operations, and managing data efficiently.

Next.js

Next.js is a modern React-based framework used for building web applications. It provides features such as fast rendering, routing, and server-side capabilities, which help in developing efficient and scalable applications.

React.js

React is a JavaScript library used for building interactive user interfaces. It allows developers to create reusable components and provides a smooth user experience for web applications.

Node.js

Node.js is used as the backend runtime environment for handling server-side operations. It allows the system to process requests, manage APIs, and interact with the database efficiently.

JavaScript

JavaScript is the primary programming language used in the development of the system. It is used for implementing both frontend and backend functionalities.

Database (JSON / Database Storage)

A database is used to store important information such as voter details, candidate information, and voting records. The stored data helps the system manage election activities and generate results accurately.

Visual Studio Code

Visual Studio Code is used as the development environment for writing and managing the project code. It provides useful features such as debugging tools, extensions, and code management support.

3.2 Resources Required

To successfully develop and run the Online Voting System, certain hardware and software resources are required.

Hardware Requirements

- Processor: Intel Core i3 or higher
- RAM: Minimum 4GB
- Storage: 100GB or higher
- Display: Standard Monitor
- Internet Connection for accessing the system

Software Requirements

- Operating System: Windows / Linux / macOS
- Programming Language: JavaScript
- Frameworks: Next.js, React.js
- Backend Environment: Node.js
- Development Tool: Visual Studio Code
- Web Browser: Google Chrome / Microsoft Edge

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces, and data flow of a system to satisfy the specified requirements. It provides a blueprint for the implementation of the system. The main objective of system design is to convert system requirements into a detailed structure that guides the development process.

The **Online Voting System** is designed as a web-based application that allows voters to cast their votes through an online platform. The system consists of several modules including voter authentication, nominee management, vote casting, vote storage, and result generation. These modules work together to ensure that the voting process is secure, transparent, and efficient.

The system design is divided into the following components:

- Architectural Design
- Input and Output Form Design
- Table Design
- Data Flow Diagram
- Database Normalization
- Entity Relationship Diagram
- Data Dictionary

4.1 Architectural Design

Architectural design describes the overall structure of the system and the interaction between different components.

The Online Voting System mainly consists of the following modules:

1. User Authentication Module
2. Voter Management Module
3. Nominee Management Module
4. Vote Casting Module
5. Vote Storage Module
6. Result Calculation Module
7. Admin Dashboard Module

In this system, voters log into the platform using their credentials. After authentication, they can view the list of candidates and cast their vote. The system stores the vote securely in the database and ensures that each voter can vote only once.

The administrator manages the election process through the admin dashboard where they can add nominees, view voter information, monitor votes, and publish election results.

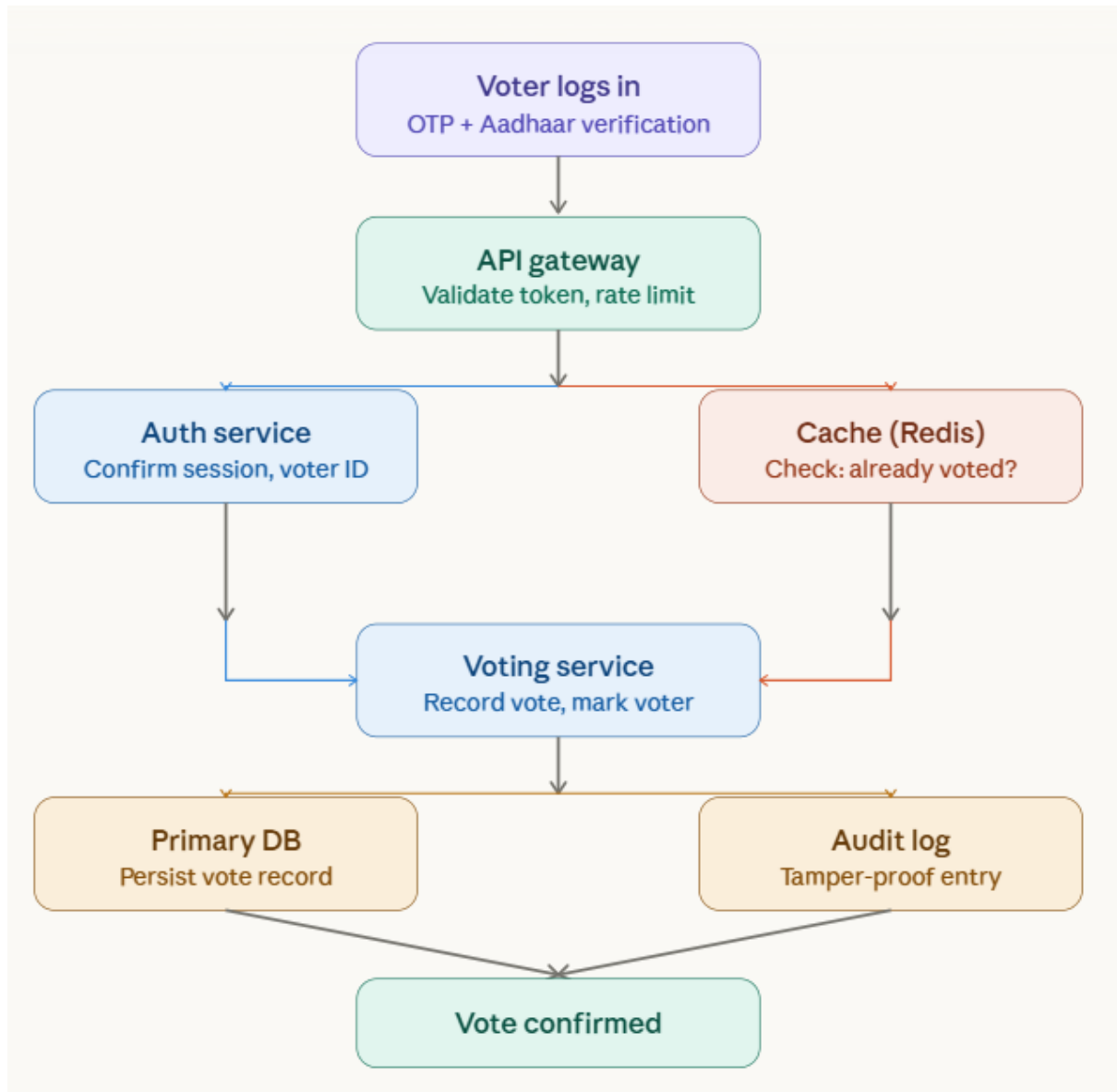


Figure 4.1: System Architecture Diagram

4.2 I/O Form Design

Input and Output design focuses on how data enters the system and how the results are displayed to users.

Input Design

The system accepts several types of input data:

- Voter login details (username / password)
- Voter registration information
- Candidate or nominee details
- Vote selection from voters

These inputs are processed by the system to perform voting operations securely.

Output Design

The system generates various outputs such as:

- List of nominees available for voting
- Confirmation message after voting
- Voting statistics
- Election result display

These outputs are displayed through the web interface for both voters and administrators.

4.3 Table Design

Table design represents the structure of the database used to store system data. The Online Voting System uses multiple tables to manage election information.

Voters Table

This table stores information about registered voters.

Attributes:

- voter_id (Primary Key)
- voter_name
- email
- password
- has_voted

Nominees Table

This table stores candidate or nominee details.

Attributes:

- nominee_id (Primary Key)

- nominee_name
- party_name
- vote_count

Votes Table

This table stores voting records.

Attributes:

- vote_id (Primary Key)
- voter_id
- nominee_id
- vote_time

4.4 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) shows how data moves through the system and how it is processed by different modules.

DFD Level 0

The Level 0 diagram provides an overview of the entire Online Voting System. It shows the interaction between external users and the system.

In this diagram:

- Voters provide login details and cast votes.
- The system processes the vote and stores it in the database.
- The administrator monitors the election process and views results.

DFD Level 1

The Level 1 DFD provides a more detailed view of the internal processes of the system.

The system is divided into several processes such as:

- Voter authentication
- Nominee display
- Vote casting
- Vote storage
- Result calculation

Each process receives input data, performs necessary operations, and produces output results.

4.5 Normalization

Database normalization is the process of organizing data in a database to reduce redundancy and improve data integrity.

The database used in the Online Voting System follows normalization principles.

First Normal Form (1NF)

Each table contains atomic values and there are no repeating groups.

Second Normal Form (2NF)

All non-key attributes depend entirely on the primary key.

Third Normal Form (3NF)

Transitive dependencies are removed so that non-key attributes depend only on the primary key.

Normalization helps maintain data consistency and improves database efficiency.

4.6 Entity Relationship Diagram (ER Diagram)

The Entity Relationship Diagram represents the relationship between different entities in the database.

The main entities in the Online Voting System include:

- Voter
- Nominee
- Vote
- Administrator

The **Voter** entity stores information about registered voters.

The **Nominee** entity stores candidate details.

The **Vote** entity records voting data.

The **Administrator** entity manages the election process.

The relationships between these entities ensure proper management of election activities.

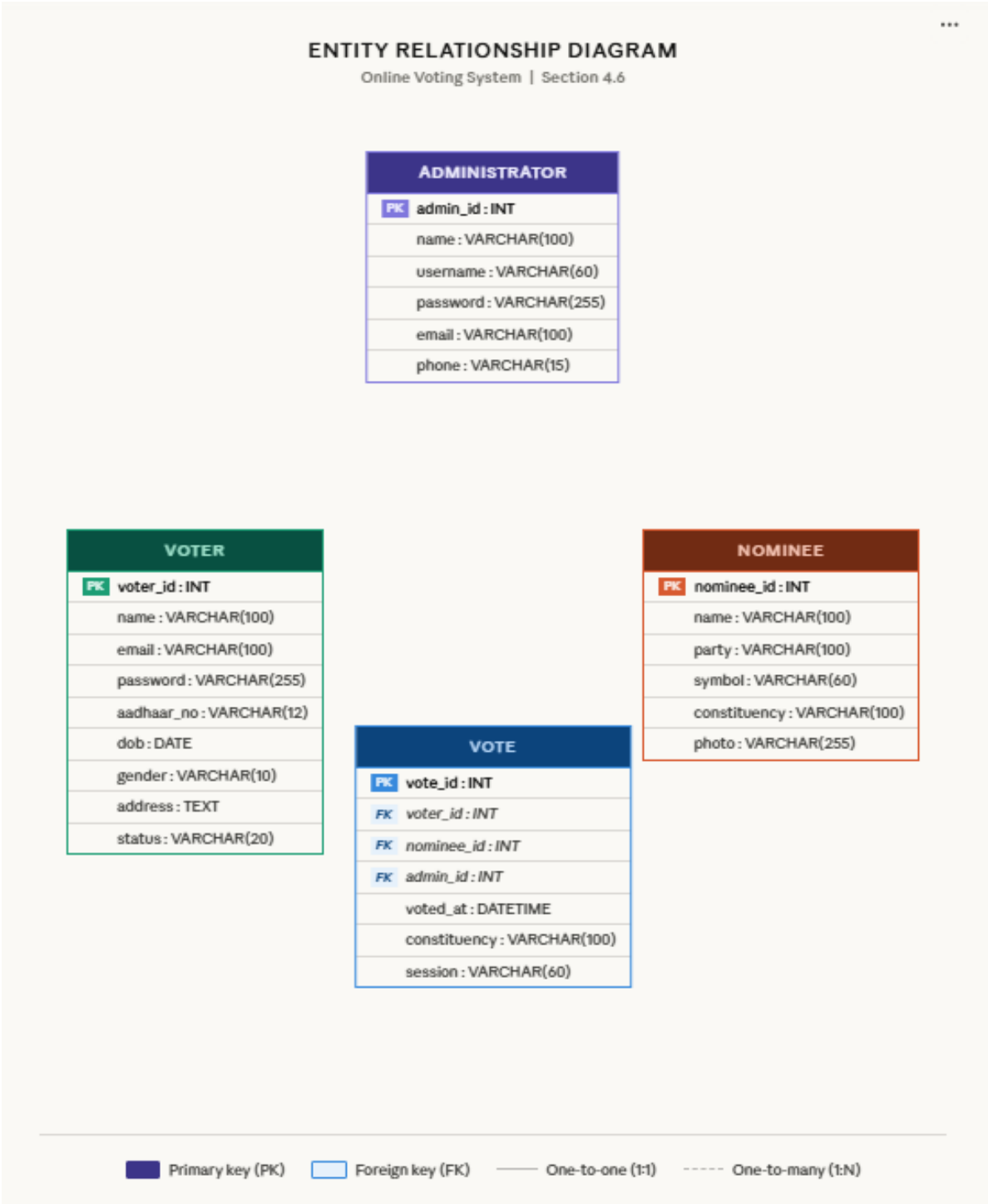


Figure 4.6 Entity Relationship Diagram

4.7 Data Dictionary

The data dictionary describes the meaning and usage of each data element in the system.

Field Name	Description
voter_id	Unique identifier for voter
voter_name	Name of the voter
email	Voter email address
password	Login password
nominee_id	Unique identifier for nominee
nominee_name	Name of the candidate
party_name	Political party name
vote_count	Total votes received
vote_id	Unique identifier for vote
vote_time	Time when vote was cast

SYSTEM DEVELOPMENT

System development is the process of implementing the system design using appropriate programming languages, frameworks, and technologies. During this phase, different modules of the system are developed and integrated to perform the required functionalities.

The **Online Voting System** is developed as a web-based application that allows voters to participate in elections through a secure digital platform. The system includes several functional modules that work together to manage the voting process efficiently. These modules handle tasks such as user authentication, nominee management, vote casting, vote storage, and result generation.

The development process ensures that the system is secure, reliable, and easy to use for both voters and administrators.

5.1 Functional Documentation

Functional documentation explains how each module in the system works and how different components interact with each other.

5.1.1 User Authentication Module

The user authentication module is responsible for verifying the identity of voters and administrators before allowing access to the system. Users must log in using their registered credentials such as username and password.

After successful authentication, the system grants access to the voting interface for voters or the admin dashboard for administrators. This module ensures that only authorized users can access the system and participate in the election process.

5.1.2 Voter Registration Module

The voter registration module allows administrators to register voters in the system. Each voter is provided with unique login credentials to access the platform.

The system stores voter details such as name, email, and login information in the database. This module ensures that only registered voters can participate in the election.

5.1.3 Nominee Management Module

The nominee management module allows the administrator to add, update, or remove candidate details from the system.

Each nominee is registered with information such as candidate name, party name, and other relevant details. The nominees added by the administrator are displayed to voters during the voting process.

5.1.4 Vote Casting Module

The vote casting module allows voters to select their preferred candidate and cast their vote through the system interface.

After logging into the system, the voter can view the list of available nominees. The voter selects one candidate and submits the vote. Once the vote is cast, the system records it in the database and prevents the voter from voting again.

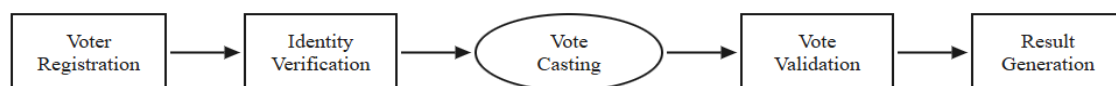


Figure 5.1.4 Vote Casting Pipeline

5.1.5 Vote Storage Module

The vote storage module is responsible for securely storing all voting records in the database.

Each vote is stored along with details such as voter ID, nominee ID, and time of voting. This module ensures data integrity and helps maintain accurate voting records for result calculation.

5.1.6 Result Calculation Module

The result calculation module automatically counts the votes received by each nominee. The system retrieves voting data from the database and calculates the total votes for each candidate. The final election results are then displayed through the admin dashboard or results page.

5.1.7 Admin Dashboard Module

The admin dashboard module provides a control panel for managing the entire election process.

Through this dashboard, the administrator can:

- Add or manage nominees
- Register or view voters
- Monitor vote counts
- View election statistics
- Publish final election results

This module allows administrators to efficiently control and manage the voting system.

5.2 Special Features of the Language

The Online Voting System is developed using modern web technologies such as JavaScript, React, and Node.js. These technologies provide flexibility, efficiency, and scalability for developing web applications.

JavaScript is used as the primary programming language for both frontend and backend development. It allows developers to build interactive and dynamic web applications.

React.js is used to create reusable user interface components and improve the user experience. It enables efficient rendering of web pages and provides a responsive interface for users.

Next.js is used as the framework for building the web application. It provides advanced features such as fast page rendering, routing, and optimized performance.

Node.js is used to handle server-side operations and manage backend logic. It allows the system to process requests, interact with the database, and manage voting data efficiently.

Overall, the selected technologies provide a strong foundation for building a secure and efficient Online Voting System.

6.TESTING

Testing is an important phase in software development where the developed system is evaluated to ensure that it works correctly and meets the specified requirements. The purpose of testing is to identify errors, verify system functionality, and ensure that the system performs efficiently under different conditions.

In the **Online Voting System**, testing was conducted to ensure that all modules of the system work properly and that the voting process is secure and reliable. Different testing methods were applied to verify the accuracy of the voting process, authentication mechanism, and result calculation.

Testing also ensures that the system can handle user interactions smoothly and that voters can cast their votes without any errors. Through proper testing, the reliability and performance of the system are improved before deployment.

The following types of testing were performed during the development of the Online Voting System:

- Unit Testing
- Integration Testing
- System Testing
- Performance Testing
- Security Testing

6.1 Types of Testing Done

Unit Testing

Unit testing is the process of testing individual modules of the system separately to ensure that each module performs its function correctly.

In the Online Voting System, the following modules were tested individually:

- User authentication module
- Voter registration module
- Nominee management module
- Vote casting module
- Result calculation module

Integration Testing

Integration testing is performed to ensure that different modules of the system work properly when combined together.

In the Online Voting System, integration testing was performed to verify the interaction between modules such as:

- Login module and voter dashboard
- Nominee management module and voting interface
- Vote casting module and database storage
- Vote storage module and result calculation module

This testing ensured that data flows correctly between modules without errors.

System Testing

System testing evaluates the complete system to verify that all functionalities operate as expected.

The Online Voting System was tested by simulating the complete election process including voter login, candidate selection, vote casting, and result generation. The system successfully recorded votes and generated results accurately.

Performance Testing

Performance testing is used to evaluate how the system performs under different conditions.

The system was tested to measure factors such as:

- Response time of the application
- Database performance
- Handling multiple users

The system maintained stable performance during testing and was able to handle voting operations efficiently.

Security Testing

Security testing is very important for an online voting system to ensure that voting data remains safe and protected.

The following security checks were performed:

- Verification of user authentication
- Prevention of duplicate voting

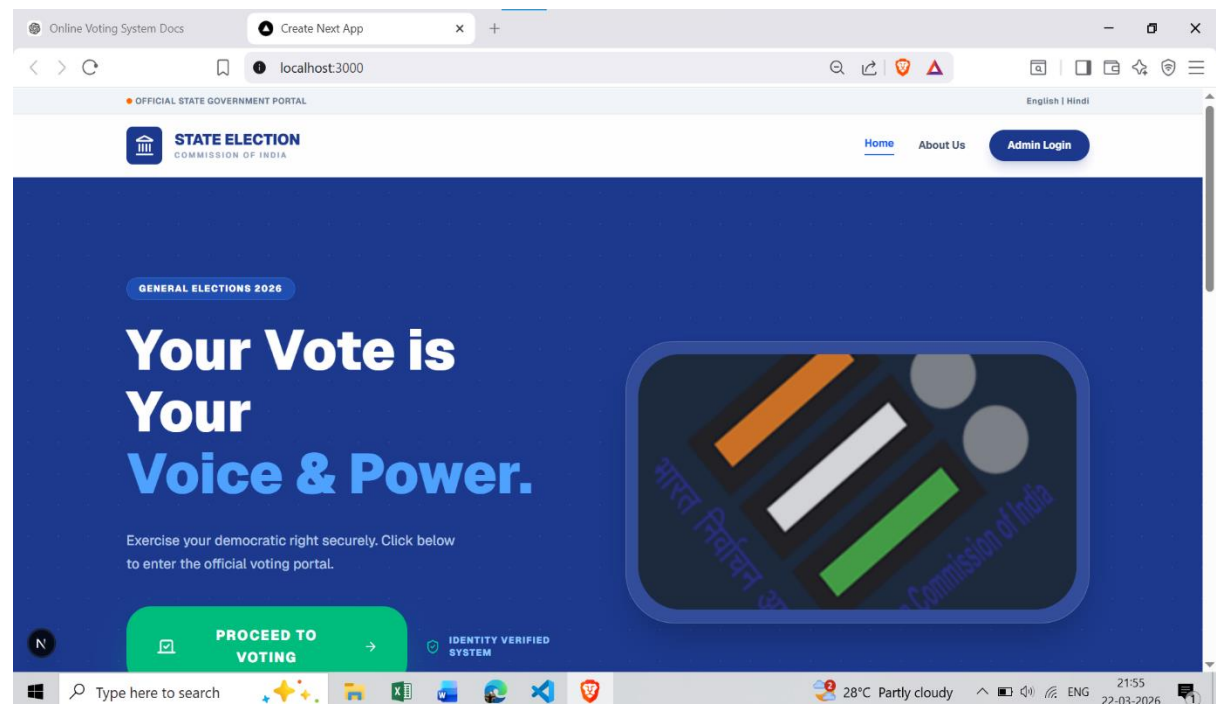
- Secure storage of voting data
- Protection against unauthorized access

These security measures ensure that the voting process remains transparent and trustworthy.

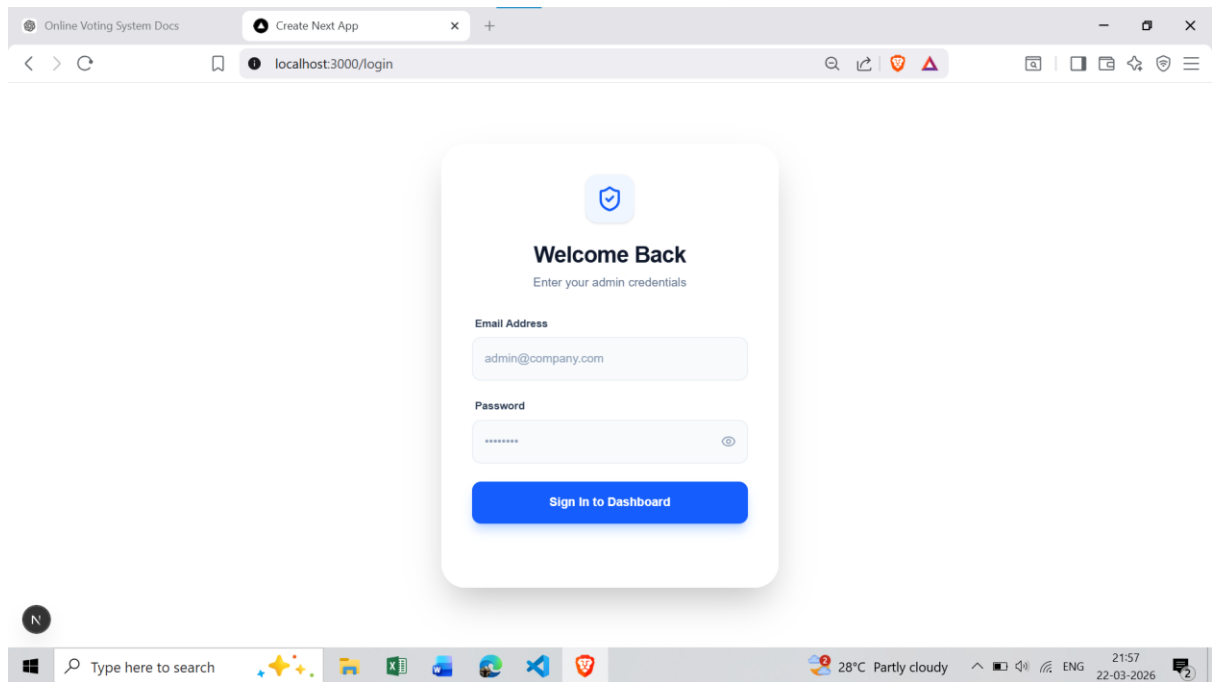
Test Result:

- The system successfully allowed registered users to log in and cast their votes without errors.
- Each voter was able to vote only once, and duplicate voting was effectively prevented.
- Votes were stored correctly in the database and the result calculation was accurate.
- The system performed efficiently with smooth user interaction and quick response time.

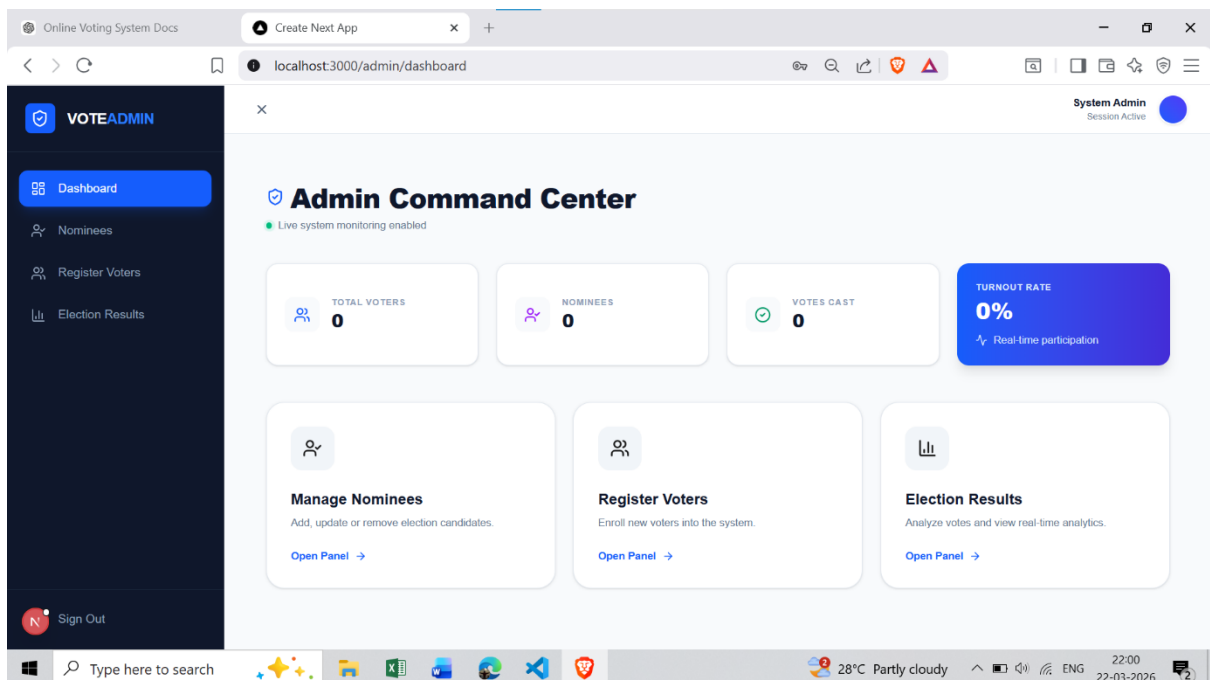
SCREENSHOTS HERE:



6.1 The Online Voting System Launching Page



6.2 The Admin of Login for the Online Voting System



6.3 The rough counting of voting in Admin login

The screenshot shows a web browser window with the URL `localhost:3000/admin/register-voters`. The page is titled "Election Commission Portal" and "Digital Voter Registration System". It features a sidebar with navigation links: Dashboard, Nominees, Register Voters (highlighted), and Election Results. The main content area contains a registration form with the following fields:

- Full Name ***: A text input field with a placeholder "Enter your full name".
- Phone Number**: A text input field with a placeholder "10 digit mobile number".
- Aadhar Number**: A text input field with a placeholder "12 digit Aadhar".
- Date of Birth ***: A date picker field with a placeholder "dd-mm-yyyy".
- Gender**: A dropdown menu with "Male" selected.
- Address**: A text input field with a placeholder "House number, street, city, pincode".

At the bottom of the form is a blue button labeled "Generate Voter ID >". The top right corner shows "System Admin" and "Session Active". The bottom of the browser window shows a Windows taskbar with a search bar and system tray icons.

6.4 SIR form in the Admin Login

The screenshot shows a web browser window with the URL `localhost:3000/admin/fetched-votes`. The page is titled "Election Audit Dashboard" and "Monitor and verify all votes in real-time". It features a sidebar with navigation links: Dashboard, Nominees, Register Voters, and Election Results (highlighted). The main content area contains the following elements:

- TOTAL VOTES**: A card showing a count of 0.
- Search**: A text input field with a placeholder "Search by voter name or voter ID..." and a button "0 record(s)".
- Voter Records Table**: A table with columns: VOTER, VOTER ID, NOMINEE, and DATE. The table is currently empty, showing a message "No voting records found".

A blue button labeled "Refresh" is located in the top right corner of the dashboard. The top right corner shows "System Admin" and "Session Active". The bottom of the browser window shows a Windows taskbar with a search bar and system tray icons.

6.5 The Realtime Counting the Election Results in the Admin Login

USER MANUAL:

The user manual provides detailed instructions on how to install, configure, and operate the Online Voting System. It helps users understand the system requirements, installation process, and how to use the application effectively.

The Online Voting System is designed to be simple, secure, and user-friendly. It allows voters to log in, view nominees, and cast their votes online, while administrators can manage the entire election process through a dashboard.

The user manual includes the following sections:

- System Requirements
- Software Installation
- Running the System
- Using the Application
- System Controls

7.1 System Requirements

Before installing the Online Voting System, the user must ensure that the system meets the minimum hardware and software requirements.

Hardware Requirements

The system requires the following hardware components:

- Processor: Intel Core i3 or higher
- RAM: Minimum 4 GB
- Storage: At least 100 GB free space
- Display: Standard monitor or laptop screen
- Internet Connection for accessing the system

Software Requirements

The following software tools are required to run the system:

- Operating System: Windows / Linux / macOS
- Programming Language: JavaScript
- Development Environment: Visual Studio Code
- Required Technologies:
- Node.js

- Next.js
- React.js

These technologies are required for frontend development, backend processing, and application execution.

7.2 Software Installation

The installation process involves setting up Node.js and installing required project dependencies.

Step 1 – Install Node.js

Download Node.js from the official website:

<https://nodejs.org>

Install Node.js and ensure it is properly configured in the system.

Step 2 – Download Project Files

Copy or download the project folder which contains all necessary files such as:

- Frontend components
- Backend files
- Configuration files
- Database files

Ensure all files are placed in the same project directory.

Step 3 – Install Dependencies

Open the project folder in Visual Studio Code and run the following command in the terminal:

```
npm install
```

This command installs all required packages for the project.

7.3 Running the System

After installing all dependencies, the system can be started using the following steps:

1. Open the project folder in Visual Studio Code
2. Run the following command in the terminal:

```
npm run dev
```

3. Open a web browser and go to:

<http://localhost:3000>

The Online Voting System interface will be displayed.

Users can now log in and access the system.

7.4 Using the Application

The system provides different interfaces for voters and administrators.

Voter Side

- Login using registered credentials
- View list of nominees
- Select a candidate and cast vote
- Receive confirmation after voting

Once the vote is cast, the voter cannot vote again.

Admin Side

- Login to admin dashboard
- Add or manage nominees
- View registered voters
- Monitor votes
- View and publish results

The admin dashboard provides full control over the election process.

7.5 System Controls

The Online Voting System includes basic controls for smooth operation:

- Login authentication to ensure secure access
- One-time voting restriction for each voter
- Real-time vote storage and result calculation
- Secure data handling to prevent unauthorized access

These controls ensure that the voting process is safe, accurate, and transparent.

7.6 System Shutdown

To stop the system:

1. Go to the terminal where the application is running
2. Press:

Ctrl + C

This will stop the server and terminate the application safely.

CONCLUSION:

8.1 Summary of the Project

The **Online Voting System** developed in this project aims to simplify and modernize the election process by providing a secure and efficient digital platform for voting.

Traditional voting methods often require significant time, manpower, and resources, and may be prone to errors and delays. The proposed system addresses these challenges by enabling online voting through a web-based application.

The system allows registered voters to log in securely and cast their votes through an easy-to-use interface. It ensures that each voter can vote only once by implementing authentication mechanisms and vote validation techniques. This helps maintain the integrity and fairness of the election process.

The project includes several modules such as voter registration, nominee management, vote casting, vote storage, and result calculation. The administrator can manage the entire election process through a dashboard, where they can add candidates, monitor voting activity, and publish results.

The system uses modern web technologies to provide a responsive and interactive user experience. All votes are stored securely in the database, and the results are calculated automatically, reducing manual effort and minimizing errors.

Additionally, the system ensures transparency and accuracy by maintaining proper records of votes and preventing unauthorized access. The testing phase confirmed that the system performs efficiently and handles user operations smoothly.

Overall, the Online Voting System provides a reliable, fast, and user-friendly solution for conducting elections in institutions and organizations. The project demonstrates how digital technology can enhance the efficiency, security, and transparency of the voting process.

8.2 Future Enhancements

Although the Online Voting System performs effectively, several enhancements can be implemented to improve its functionality and scalability.

One possible enhancement is the integration of advanced authentication methods such as One-Time Password (OTP) or biometric verification to further strengthen system security and prevent unauthorized access.

Another improvement would be the implementation of blockchain technology to ensure complete transparency and tamper-proof vote storage. This would increase trust in the election process.

The system can also be enhanced by developing a mobile application version, allowing users to vote easily through smartphones. This would improve accessibility and user convenience. Additionally, the system can be scaled to support large-scale elections with a higher number of users and improved performance optimization.

Advanced analytics and reporting features can also be added to provide detailed insights into voting patterns and election statistics.

With these enhancements, the Online Voting System can become a more powerful and secure platform suitable for large-scale and real-world election scenarios.

BIBLIOGRAPHY

Book References

1. Kogent Learning Solutions Inc. – *Web Technologies: HTML, JavaScript, PHP, Java, JSP, XML and AJAX*
2. Achyut S. Godbole & Atul Kahate – *Web Technologies*
3. E. Balagurusamy – *Programming with JavaScript*

Website References

1. <https://react.dev>
2. <https://nextjs.org>
3. <https://nodejs.org>
4. <https://developer.mozilla.org>
5. <https://www.w3schools.com>

APPENDIX

(a)Program Coding

Dashboard frontend code :

```
"use client";

import { useEffect, useState } from "react";
import { useRouter } from "next/navigation";
import { motion } from "framer-motion";
import { getToken } from "../../utils/auth";
import {
  UserCheck,
  Users,
  BarChart3,
  ArrowRight,
  CheckCircle2,
  Activity,
  ShieldCheck
} from "lucide-react";
export default function Dashboard() {
  const router = useRouter();
  const [loading, setLoading] = useState(true);
  const [stats, setStats] = useState({
    voters: 0,
    nominees: 0,
    votesCast: 0,
    participation: 0
  });
  /* ----- VERIFY + FETCH ----- */
  useEffect(() => {
    const token = getToken();
    if (!token) {
      router.push("/admin/login");
      return;
    }
    const fetchStats = async () => {
      try {
        const res = await fetch("/api/admin/stats", {
          cache: "no-store"
        });
        const data = await res.json();
        if (data.success) setStats(data.stats);
      } catch (err) {
        console.error("Stats fetch failed", err);
      }
    };
    const initDashboard = async () => {
      const verify = await fetch("/api/admin/verify", {
        headers: { Authorization: `Bearer ${token}` }
      });
      if (!verify.ok) {
        router.push("/admin/login");
      } else {

```

```

        await fetchStats();
        setLoading(false);
    }
};
initDashboard();
const interval = setInterval(fetchStats, 30000);
return () => clearInterval(interval);
}, [router]);
/* ----- LOADING ----- */
if (loading) {
    return (
        <div className="h-[80vh] flex flex-col items-center justify-center gap-4">
            <div className="w-12 h-12 border-4 border-blue-600 border-t-transparent rounded-full
animate-spin"/>
            <p className="text-slate-500 font-medium animate-pulse">
                Loading Admin Command Center...
            </p>
        </div>
    );
}
/* ----- STATS ----- */
const statItems = [
    {
        label: "Total Voters",
        value: stats.voters,
        icon: <Users size={22}/>,
        color: "text-blue-600"
    },
    {
        label: "Nominees",
        value: stats.nominees,
        icon: <UserCheck size={22}/>,
        color: "text-purple-600"
    },
    {
        label: "Votes Cast",
        value: stats.votesCast,
        icon: <CheckCircle2 size={22}/>,
        color: "text-emerald-600"
    }
];
/* ----- NAVIGATION CARDS ----- */
const cards = [
    {
        title: "Manage Nominees",
        desc: "Add, update or remove election candidates.",
        icon: <UserCheck size={26}/>,
        path: "/admin/nominee"
    },
    {
        title: "Register Voters",
        desc: "Enroll new voters into the system.",
        icon: <Users size={26}/>,
        path: "/admin/register-voters"
    },
];

```



```

    {
      title: "Election Results",
      desc: "Analyze votes and view real-time analytics.",
      icon: <BarChart3 size={26}/>,
      path: "/admin/fetched-votes"
    }
  ];
  return (
    <motion.div
      initial={{ opacity: 0 }}
      animate={{ opacity: 1 }}
      className="max-w-7xl mx-auto px-6 py-8"
    >
      { /* HEADER */ }
      <div className="flex flex-col md:flex-row md:items-center justify-between mb-10 gap-4">
        <div>
          <h1 className="text-3xl md:text-4xl font-extrabold text-slate-900 tracking-tight flex items-center gap-2">
            <ShieldCheck className="text-blue-600"/>
            Admin Command Center
          </h1>
          <p className="text-slate-500 text-sm mt-2 flex items-center gap-2">
            <span className="relative flex h-2 w-2">
              <span className="animate-ping absolute inline-flex h-full w-full rounded-full bg-emerald-400 opacity-75"></span>
              <span className="relative inline-flex rounded-full h-2 w-2 bg-emerald-500"></span>
            </span>
            Live system monitoring enabled
          </p>
        </div>
      </div>
      { /* STATS */ }
      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6 mb-12">
        {statItems.map((stat, i) => (
          <motion.div
            key={stat.label}
            initial={{ opacity: 0, y: 15 }}
            animate={{ opacity: 1, y: 0 }}
            transition={{ delay: i * 0.1 }}
            className="bg-white border border-slate-200 rounded-2xl p-6 shadow-sm flex items-center gap-4"
          >
            <div className={`p-3 rounded-xl bg-slate-50 ${stat.color}`}>
              {stat.icon}
            </div>
            <div>
              <p className="text-xs uppercase tracking-wider text-slate-400 font-bold">
                {stat.label}
              </p>
              <p className="text-2xl font-extrabold text-slate-900">
                {stat.value}
              </p>
            </div>
          </motion.div>
        ))}
      </div>

```

```

    { /* TURNOUT */ }
    <div className="bg-gradient-to-r from-blue-600 to-indigo-700 rounded-2xl p-6 text-white
shadow-lg">
      <p className="text-xs uppercase tracking-wider text-blue-100 font-bold">
        Turnout Rate
      </p>
      <p className="text-3xl font-extrabold mt-1">
        {stats.participation}%
      </p>
      <div className="mt-3 flex items-center gap-2 text-blue-100 text-sm">
        <Activity size={16}/>
        Real-time participation
      </div>
    </div>
  </div>
  { /* ACTION CARDS */ }
  <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
    {cards.map((card, idx) > (
      <motion.div
        key={card.title}
        whileHover={{ y: -6 }}
        transition={{ type: "spring", stiffness: 200 }}
        onClick={() => router.push(card.path)}
        className="group bg-white border border-slate-200 rounded-3xl p-8 shadow-sm
hover:shadow-xl transition-all cursor-pointer"
      >
        <div className="mb-6 inline-flex p-4 rounded-2xl bg-slate-100 group-hover:bg-blue-600
group-hover:text-white transition">
          {card.icon}
        </div>
        <h2 className="text-xl font-bold text-slate-900 mb-2">
          {card.title}
        </h2>
        <p className="text-slate-500 text-sm mb-6">
          {card.desc}
        </p>
        <div className="flex items-center text-sm font-bold text-blue-600">
          Open Panel
          <ArrowRight
            size={16}
            className="ml-2 group-hover:translate-x-1 transition"
          />
        </div>
      </motion.div>
    )})}
  </div>
);
}

```

Votting Frontend Code:

```

"use client";
import { useState, useEffect } from "react";
import { useRouter } from "next/navigation";

```

```

import {
  Vote,
  CheckCircle,
  AlertCircle,
  UserCheck,
  FileCheck,
  ShieldCheck
} from "lucide-react";
import { motion } from "framer-motion";
import Header from "../components/Header";
interface Nominee {
  _id: string;
  name: string;
  party: string;
}
export default function VotingWizard() {
  const router = useRouter();
  const [step, setStep] = useState(1);
  const [voterIdInput, setVoterIdInput] = useState("");
  const [nominees, setNominees] = useState<Nominee[]>([]);
  const [selectedNominee, setSelectedNominee] = useState<Nominee | null>(null);
  const [status, setStatus] = useState({ type: "", msg: "" });
  const [loading, setLoading] = useState(false);
  useEffect(() => {
    fetch("/api/nominees")
      .then((res) => res.json())
      .then((data) => setNominees(data.data || []))
      .catch(() => {
        setStatus({
          type: "error",
          msg: "Unable to load nominees"
        });
      });
  }, []);
  const next = () => setStep((s) => s + 1);
  const back = () => setStep((s) => s - 1);
  const submitVote = async () => {
    setLoading(true);
    try {
      const res = await fetch("/api/vote", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          voterIdString: voterIdInput.trim(),
          nomineeId: selectedNominee!._id
        })
      });
      const data = await res.json();
      if (res.ok) setStep(4);
      else setStatus({ type: "error", msg: data.message });
    } catch {
      setStatus({
        type: "error",
        msg: "Server connection failed"
      });
    }
  };
}

```

```

    } finally {
      setLoading(false);
    }
  };
  return (
    <>
      <Header />
      <div className="min-h-screen bg-gradient-to-br from-slate-950 via-blue-950 to-slate-900 py-16
px-4">
        <div className="max-w-3xl mx-auto">
          <motion.div
            initial={{ opacity: 0, y: 40 }}
            animate={{ opacity: 1, y: 0 }}
            className="bg-white/95 backdrop-blur-xl rounded-3xl shadow-2xl overflow-hidden"
          >
            {/* HEADER */}
            <div className="bg-gradient-to-r from-blue-900 to-slate-900 text-white p-8">
              <h1 className="text-3xl font-bold flex items-center gap-3">
                <Vote size={28} />
                National Voting Portal
              </h1>
              <p className="text-blue-200 mt-2 text-sm">
                Secure Digital Election System
              </p>
            </div>
            {/* BODY */}
            <div className="p-10 space-y-8">
              {status.msg && (
                <div className="flex gap-3 p-4 bg-red-50 border border-red-200 text-red-700 rounded-xl">
                  <AlertCircle />
                  {status.msg}
                </div>
              )}
              {/* STEP 1 */}
              {step === 1 && (
                <div className="space-y-6">
                  <h2 className="text-2xl font-bold text-gray-800">
                    Voter Identity Verification
                  </h2>
                  <input
                    className="w-full p-4 border-2 border-gray-200 rounded-xl
                    focus:ring-2 focus:ring-blue-300 focus:border-blue-500 outline-none
                    text-black font-mono"
                    placeholder="Enter Voter ID"
                    value={voterIdInput}
                    onChange={(e) =>
                      setVoterIdInput(e.target.value.toUpperCase())
                    }
                  />
                  <button
                    onClick={next}
                    disabled={!voterIdInput.trim()}
                    className="w-full py-4 bg-blue-600 text-white font-bold rounded-xl hover:bg-blue-700"
                  >
                    Continue

```

```

        </button>
    </div>
)}
{/* STEP 2 */}
{step === 2 && (
    <div className="space-y-4">
        <h2 className="text-xl font-bold text-gray-800">
            Choose Your Candidate
        </h2>
        {nominees.map((n) => (
            <div
                key={n._id}
                onClick={() => setSelectedNominee(n)}
                className={`p-5 rounded-xl border cursor-pointer flex justify-between
                    ${
                        selectedNominee?._id === n._id
                        ? "border-blue-600 bg-blue-50"
                        : "border-gray-200"
                    }
                `}
            >
                <div>
                    <p className="font-bold text-gray-800">
                        {n.name}
                    </p>
                    <p className="text-blue-600 text-sm">
                        {n.party}
                    </p>
                </div>
                {selectedNominee?._id === n._id && (
                    <CheckCircle className="text-blue-600" />
                )}
            </div>
        ))}
        <div className="flex gap-4 pt-4">
            <button
                onClick={back}
                className="flex-1 py-3 bg-gray-200 rounded-xl"
            >
                Back
            </button>
            <button
                onClick={next}
                disabled={!selectedNominee}
                className="flex-1 py-3 bg-blue-600 text-white rounded-xl"
            >
                Review
            </button>
        </div>
    </div>
)}
{/* STEP 3 */}
{step === 3 && selectedNominee && (
    <div className="text-center space-y-6">
        <h2 className="text-2xl font-bold">
            Confirm Your Vote

```

```

    </h2>
    <div className="bg-gray-50 p-6 rounded-xl border">
      <p className="font-mono">{voterIdInput}</p>
      <p className="text-xl font-bold text-blue-700">
        {selectedNominee.name}
      </p>
      <p className="text-blue-600">
        {selectedNominee.party}
      </p>
    </div>
    <button
      onClick={submitVote}
      disabled={loading}
      className="w-full py-4 bg-green-600 text-white font-bold rounded-xl hover:bg-green-
700"
    >
      {loading ? "Submitting Vote..." : "Cast Vote Securely"}
    </button>
  </div>
)}
{/* SUCCESS */}
{step === 4 && (
  <div className="text-center space-y-6">
    <CheckCircle className="w-20 h-20 text-green-500 mx-auto" />
    <h2 className="text-3xl font-bold text-gray-800">
      Vote Successfully Recorded
    </h2>
    <p className="text-gray-500">
      Thank you for strengthening democracy.
    </p>
    <button
      onClick={() => router.push("/")}
      className="px-8 py-4 bg-blue-900 text-white rounded-xl font-bold"
    >
      Return Home
    </button>
  </div>
)}
</div>
</motion.div>
{/* ELECTION COMMISSION QUOTES */}
<div className="mt-10 text-center text-blue-200 text-sm space-y-2">
  <p>IN “My Vote is My Future – Power of One Vote.”</p>
  <p>IN “No Voter to be Left Behind.”</p>
  <p>IN “Electoral Literacy for Stronger Democracy.”</p>
</div>
{/* SECURITY */}
<p className="text-center text-blue-300 text-xs mt-4 flex justify-center gap-2 items-center">
  <ShieldCheck size={14} />
  All votes are encrypted and protected.
</p>
</div>
</div>
</>

```

```
);  
}
```

Fetches-Votes in the frontend

```
"use client";  
import { useEffect, useState } from "react";  
import { motion, AnimatePresence } from "framer-motion";  
import {  
  Search,  
  RefreshCw,  
  UserCheck,  
  Calendar,  
  FileText,  
} from "lucide-react";  
interface Vote {  
  _id: string;  
  voter: {  
    name: string;  
    phone: string;  
    voterId: string;  
  };  
  nominee: {  
    name: string;  
    party: string;  
  };  
  createdAt: string;  
}  
export default function VotesPage() {  
  const [votes, setVotes] = useState<Vote[]>([]);  
  const [loading, setLoading] = useState(true);  
  const [searchTerm, setSearchTerm] = useState("");  
  const fetchVotes = async () => {  
    setLoading(true);  
    try {  
      const res = await fetch("/api/admin/fetch-votes");  
      const data = await res.json();  
      setVotes(data.votes || []);  
    } catch (error) {  
      console.error("Error fetching votes:", error);  
      setVotes([]);  
    } finally {  
      setLoading(false);  
    }  
  };  
  useEffect(() => {  
    fetchVotes();  
  }, []);  
  const filteredVotes = votes.filter(  
    (vote) =>  
      vote.voter?.name?.toLowerCase().includes(searchTerm.toLowerCase()) ||  
      vote.voter?.voterId?.toLowerCase().includes(searchTerm.toLowerCase())  
  );  
  return ( <div className="min-h-screen bg-gradient-to-br from-slate-100 via-white to-blue-100 p-6  
md:p-12">
```

```

<div className="max-w-7xl mx-auto">
  {/* HEADER */}
  <div className="flex flex-col md:flex-row justify-between items-center mb-10 gap-6">
    <div>
      <h1 className="text-4xl font-extrabold text-slate-900">
        Election Audit Dashboard
      </h1>
      <p className="text-slate-600 mt-1">
        Monitor and verify all votes in real-time
      </p>
    </div>
    <button
      onClick={fetchVotes}
      className="flex items-center gap-2 px-5 py-3 bg-blue-600 text-white rounded-xl shadow
        hover:bg-blue-700 transition"
    >
      <RefreshCw size={18} className={loading ? "animate-spin" : ""} />
      Refresh
    </button>
  </div>
  {/* STATS */}
  <div className="mb-8">
    <div className="bg-white p-6 rounded-2xl shadow border flex items-center gap-5 w-fit">
      <div className="bg-blue-100 text-blue-700 w-12 h-12 flex items-center justify-center
        rounded-xl">
        <UserCheck size={26}/>
      </div>
      <div>
        <p className="text-sm text-gray-500 font-semibold">
          TOTAL VOTES
        </p>
        <p className="text-3xl font-bold text-gray-900">
          {votes.length}
        </p>
      </div>
    </div>
  </div>
  {/* SEARCH */}
  <div className="bg-white p-6 rounded-2xl shadow border mb-6 flex flex-col md:flex-row gap-
4 justify-between">
    <div className="relative w-full md:max-w-md">
      <Search
        className="absolute left-3 top-3 text-gray-400"
        size={18}
      />
      <input
        type="text"
        placeholder="Search by voter name or voter ID..."
        className="pl-10 pr-4 py-2 w-full border rounded-lg focus:ring-2 focus:ring-blue-500
        outline-none"
        value={searchTerm}
        onChange={(e) => setSearchTerm(e.target.value)}
      />
    </div>
    <p className="text-gray-500 font-medium">

```



```

    {filteredVotes.length} record(s)
  </p>
</div>
{/* TABLE */}
<div className="bg-white rounded-2xl shadow border overflow-hidden">
  <table className="w-full text-left">
    <thead className="bg-gray-100 text-gray-600 text-sm uppercase">
      <tr>
        <th className="p-4">Voter</th>
        <th className="p-4">Voter ID</th>
        <th className="p-4">Nominee</th>
        <th className="p-4">Date</th>
      </tr>
    </thead>
    <tbody>
      <AnimatePresence>
        {filteredVotes.map((vote, i) => (
          <motion.tr
            key={vote._id}
            initial={{opacity:0,y:10}}
            animate={{opacity:1,y:0}}
            transition={{delay:i*0.03}}
            className="border-t hover:bg-blue-50 transition"
          >
            <td className="p-4">
              <p className="font-semibold text-gray-900">
                {vote.voter?.name}
              </p>
              <p className="text-sm text-gray-500">
                {vote.voter?.phone}
              </p>
            </td>
            <td className="p-4 font-mono">
              {vote.voter?.voterId}
            </td>
            <td className="p-4">
              <p className="font-semibold text-blue-700">
                {vote.nominee?.name}
              </p>
              <p className="text-sm text-gray-500">
                {vote.nominee?.party}
              </p>
            </td>
            <td className="p-4 text-gray-600 flex items-center gap-2">
              <Calendar size={14}/>
              {new Date(vote.createdAt).toLocaleDateString()}
            </td>
          </motion.tr>
        )))
      </AnimatePresence>
    </tbody>
  </table>
  {/* EMPTY STATE */}
  {filteredVotes.length === 0 && !loading && (

```

```
<div className="text-center py-20 text-gray-500">
  <FileText size={40} className="mx-auto mb-3 opacity-40"/>
  No voting records found
</div>
)}
</div>
</div>
</div>
);
}
```